

Story

You and your friends are planning a multi-day group trip. Every member has a **different daily budget**, **different interests**, and a **different energy level**. The real world is messy — people drop out, the weather turns bad, some travellers get tired, and budgets get cut at the last minute.

You must build the brain behind a group-trip app that:

1. Picks the **best joint itinerary** that respects **every** participant.
2. **Re-plans** the remainder of the trip every time something changes.
3. Is **fully deterministic** so its decisions can be reviewed and audited.

Your program will be judged on whether its output exactly matches the reference planner under the rules below.

Problem

You are given a group of N travellers going on a D -day trip with H usable hours per day. You are also given a catalogue of A candidate activities and a chronological list of E real-world events that will interrupt the trip.

Definitions

- **Interest tags** are drawn from the fixed set `{ADVENTURE, CULTURE, FOOD, NATURE, SHOPPING, NIGHTLIFE}`.
- Each traveller has a **daily budget** (max spend per person per day), an **energy level** (0–100), and a non-empty set of liked tags.
- Each activity has a positive **cost per person**, an integer **duration** in hours, an integer **energy cost**, and exactly one **tag**.
- An activity is **eligible on day** d if (a) it has not been chosen on a previous day, and (b) its tag is not weather-blocked on day d .

Feasibility Constraints (The "Fairness" Rules)

Before calculating scores, you must identify all possible subsets of eligible activities that satisfy three strict limits. These limits are "bottlenecked" by the most restricted member of the group:

- **Financial Fairness:**

The total cost of the day's activities cannot exceed the daily budget of the traveler with the lowest budget.

$$\sum \text{cost}(a) \leq \min(u.\text{budget for } u \text{ in } U)$$

- **Stamina Fairness:**

The total energy cost cannot exceed the current energy level of the weakest (or most tired) active traveler.

$$\sum \text{energy}(a) \leq \min(u.\text{energy for } u \text{ in } U)$$

- **Time Constraint:**

The total hours spent on activities must fit within the fixed daily limit H .

$$\sum \text{duration}(a) \leq H$$

The Objective: Satisfaction Score

The goal is to maximize the collective enjoyment of the group. The satisfaction score for a subset S is calculated as:

$$\text{satisfaction}(S) = \sum \text{ over } a \text{ in } S \text{ of } |\{ u \in U : \text{tag}(a) \in u.\text{interests} \}|$$

In plain terms: For every activity in your proposed plan, count how many people in the group actually like that activity's category (tag). Sum these counts across all activities in the subset.

Example:

If you pick a "Hike" (NATURE) and 3 out of 5 travelers have NATURE in their interests, that activity adds 3 points to the day's satisfaction score.

The Decision Engine (Lexicographical Tie-Breaking)

Because multiple different combinations of activities might yield the same satisfaction or fit the same constraints, the problem requires a deterministic way to pick exactly one "best" subset. You must compare subsets using a tuple and pick the one that is "smallest" lexicographically:

1. **Primary Priority (Satisfaction):** Maximize the satisfaction score. (In the tuple, this is represented as `-satisfaction` because the rule seeks the "smallest" value, and a larger satisfaction becomes a smaller negative number).
2. **Secondary Priority (Cost):** If two subsets provide the same satisfaction, choose the one with the lower total cost.
3. **Tertiary Priority (ID List):** If satisfaction and cost are identical, sort the activity IDs in each subset and compare the lists. Choose the subset whose ID list comes first lexicographically (e.g., `[1, 4]` is smaller than `[2, 3]`).

Operational Logic

- **Rest Days:** If no activities can fit the constraints, or if the "best" valid subset is an empty set, the day is designated as `REST`.
- **State Persistence:** Once an activity is chosen for a day, it is removed from the "eligible" pool for all future days.

- **Replanning:** Because events (like someone dropping out or a budget change) alter the constraints (`min(u.budget)` or `min(u.energy)`), the optimization must be re-run for all remaining days whenever an event occurs.

Example

Available Activities

ID	Tag	Cost	Energy	Duration	Interested Users	Points
A1	FOOD	\$20	20	3	Alice, Bob	2
A2	NATURE	\$25	30	4	Alice	1
A3	CULTURE	\$30	40	4	Bob	1

Evaluating Possible Subsets (S):

Subset S	Total Cost	Total Energy	Total Duration	Feasible?	Satisfaction
{A1, A2}	\$45	50	7	Yes	2 + 1 = 3
{A1, A3}	\$50	60	7	Yes	2 + 1 = 3
{A2, A3}	\$55	70	8	No (Cost > \$50)	-
{A1, A2, A3}	\$75	90	11	No (Exceeds all)	-

Applying the Lexicographical Rule

We compare the two feasible subsets, {A1, A2} and {A1, A3}:

- **Satisfaction:** Both have a score of 3. (Tie)
- **Total Cost:** {A1, A2} costs ~~45~~. ~~A1, A3~~ costs 50.
- **Decision:** {A1, A2} is chosen because it is cheaper (~~45~~ < 50).

Result: The plan for the day is `[A1, A2]`. If no subsets were feasible (e.g., if all activities cost more than \$50), the output would be `REST`.

Input Format

The input consists of several sections, each on its own line or group of lines. All values are separated by spaces.

```
N D H
<userLine>      × N
A
<activityLine>  × A
E
<eventLine>     × E
```

Where: - `N` is the number of travellers ($3 \leq N \leq 10$) - `D` is the number of days ($1 \leq D \leq 7$) - `H` is the number of usable hours per day ($1 \leq H \leq 24$) - Each `<userLine>` describes one traveller - `A` is the number of activities ($1 \leq A \leq 20$) - Each `<activityLine>` describes one activity - `E` is the number of events ($0 \leq E \leq 20$) - Each `<eventLine>` describes one event

User Line Example

```
Alice 100 80 2 ADVENTURE FOOD
```

- `name` : Alice
- `dailyBudget` : 100
- `energy` : 80
- `k` : 2 (number of interest tags)
- `tags` : ADVENTURE FOOD

Activity Line Example

```
1 Museum 30 3 20 CULTURE
```

- `id` : 1
- `name` : Museum
- `cost` : 30
- `duration` : 3
- `energy` : 20
- `tag` : CULTURE

Event Line Examples

```
WEATHER 2 ADVENTURE
DROP 3 Bob
FATIGUE 2 Alice 50
BUDGET 4 Cara 60
```

- WEATHER: On day 2, all activities with tag ADVENTURE are blocked
- DROP: Bob leaves the trip starting on day 3
- FATIGUE: Alice's energy is set to 50 from day 2 onward
- BUDGET: Cara's daily budget is set to 60 from day 4 onward

Full Example Input

```
3 2 8
Alice 100 80 2 ADVENTURE FOOD
Bob 80 60 2 CULTURE FOOD
Cara 120 70 2 NATURE FOOD
4
1 Museum 30 3 20 CULTURE
2 Hike 40 5 50 ADVENTURE
3 Cafe 20 2 10 FOOD
4 Park 25 3 15 NATURE
1
WEATHER 2 ADVENTURE
```

This input describes a 3-person, 2-day trip with 8 hours per day, 4 activities, and 1 event (weather blocks ADVENTURE on day 2).

Constraints

```
3 ≤ N ≤ 10
1 ≤ D ≤ 7
1 ≤ H ≤ 24
1 ≤ A ≤ 20
0 ≤ E ≤ 20
1 ≤ cost, duration, energy, budget ≤ 10000
0 ≤ user.energy ≤ 100
```

Names are alphanumeric strings without spaces.

Output Format

Print the **initial plan** first:

```
=== PLAN ===
Day 1: <ids ...>|REST | cost=<c> satisfaction=<s>
Day 2: ...
...
Day D: ...
```

Then, for each event in order (1-indexed), print:

```
=== EVENT <i>: <original event line verbatim> ===
Day <day>: ...
...
Day D: ...
```

Activity ids on a day line are printed **in ascending order**, separated by single spaces. Use the literal word **REST** when the chosen subset is empty. **cost** is the per-person cost for that day; **satisfaction** is the integer satisfaction score.

There must be **no trailing whitespace** and the file must end with a single newline.

Example Input

```
3 2 8
Alice 100 80 2 ADVENTURE FOOD
Bob 80 60 2 CULTURE FOOD
Cara 120 70 2 NATURE FOOD
4
1 Museum 30 3 20 CULTURE
2 Hike 40 5 50 ADVENTURE
3 Cafe 20 2 10 FOOD
4 Park 25 3 15 NATURE
1
WEATHER 2 ADVENTURE
```

Output

```
=== PLAN ===
Day 1: 2 3 | cost=60 satisfaction=4
Day 2: 1 4 | cost=55 satisfaction=2
=== EVENT 1: WEATHER 2 ADVENTURE ===
Day 2: 1 4 | cost=55 satisfaction=2
```

Notes on the example

- On Day 1, picking {Hike, Cafe} scores $1(\text{Adv} \rightarrow \text{Alice}) + 3(\text{Food} \rightarrow \text{all}) = 4$, costs 60, fits energy $60 \leq \min(80, 60, 70) = 60$, fits time $7 \leq 8$. Any other feasible subset scores ≤ 4 or costs more.
- On Day 2, {Museum, Park} scores $1+1=2$, costs 55. Hike was already used.
- The weather event blocks ADVENTURE only on day 2, but Hike was already consumed on day 1, so the replan reproduces the same Day 2.

Sample Input 0

```
3 2 8
Alice 100 80 2 ADVENTURE FOOD
Bob 80 60 2 CULTURE FOOD
Cara 120 70 2 NATURE FOOD
4
1 Museum 30 3 20 CULTURE
2 Hike 40 5 50 ADVENTURE
3 Cafe 20 2 10 FOOD
4 Park 25 3 15 NATURE
1
WEATHER 2 ADVENTURE
```

Sample Output 0

```
=== PLAN ===
Day 1: 1 3 4 | cost=75 satisfaction=5
Day 2: 2 | cost=40 satisfaction=1
=== EVENT 1: WEATHER 2 ADVENTURE ===
Day 2: REST | cost=0 satisfaction=0
```